11. Testing and Evaluation Plan

The Process Change Impact Simulator will be tested to check whether its functions work correctly and meet the requirements defined earlier. I will use unit testing, integration testing, system testing, API testing, usability testing, and performance testing. The tests will use synthetic business workflows, including purchase approval and employee onboarding.

11.1 Testing Approaches

Testing approach	What will be tested	Evidence of success
Unit testing	Graph construction, downstream activity detection, cycle detection, and simulation comparison.	All defined unit tests pass with expected results.
Integration testing	Communication between Flask, SQLite, NetworkX, and the simulation engine.	Process data is saved, retrieved, analyzed, and displayed correctly.
System testing	Complete workflow from process creation to simulation results.	Users can complete the main process without application errors.
API testing	Process impact analysis endpoints and invalid requests.	Valid requests return correct JSON results, and invalid requests return suitable error messages.
Usability testing	Process creation, change selection, and result review.	Test users can complete the main tasks without programming assistance.
Performance testing	Graph analysis and simulation response times.	Analysis completes within 3 seconds and simulation comparison within 5 seconds for workflows containing up to 500 activities.
Reliability testing	Successful processing of valid test scenarios.	At least 95% of valid test scenarios complete successfully.
Data integrity testing	Saving a proposed change without modifying the original process.	Original process data remains unchanged in all tested scenarios.
Compatibility testing	Application operation in Chrome, Edge, and Firefox.	Main functions work correctly in each supported browser.
Acceptance testing	Functional requirements and expected user tasks.	All required core functions pass their defined acceptance tests.

11.2 Functional Test Cases

Test ID	Test scenario	Expected result
TC-01	Create and save a purchase approval process.	Process details are saved and displayed correctly.
TC-02	Select an activity and run dependency analysis.	All reachable downstream activities are identified correctly.

TC-03	Add a circular dependency.	System detects the cycle and displays a warning.
TC-04	Change an approval threshold.	Simulation follows the configured conditions and identifies differences in execution paths.
TC-05	Import an invalid JSON file.	System displays an error and does not save invalid process data.
TC-06	Compare original and modified processes.	Added activities, skipped activities, and output differences are reported correctly.
TC-07	Export a process as JSON.	Exported file contains the expected process activities and dependencies.
TC-08	Submit an unknown activity code to the API.	System returns an appropriate error response.

11.3 Evaluation and Evidence

I will use automated tests with `pytest` and test the web interface manually. I will record response times, compare actual results with expected results, and capture screenshots of successful tests. I will also use sample workflows with sequential activities, branching paths, and multiple approvals to check different situations.

Evaluation target	Evidence required
Unit test success	Test report showing passed and failed tests.
Performance targets	Recorded analysis and simulation response times.
Data integrity	Before-and-after database records showing that the original process remains unchanged.
Dependency accuracy	Comparison of expected downstream activities with actual results.
User interface operation	Screenshots showing process setup, analysis, and simulation results.

Testing will be considered successful when the defined acceptance criteria are met and the results are supported by test reports, measured response times, and screenshots. Any failed tests will be recorded, corrected, and repeated. These are planned evaluation targets; actual results will be reported after testing is completed.